

VLSI Practice 02 Report

2022171005 권민찬

I. 8-to-1 Multiplexer

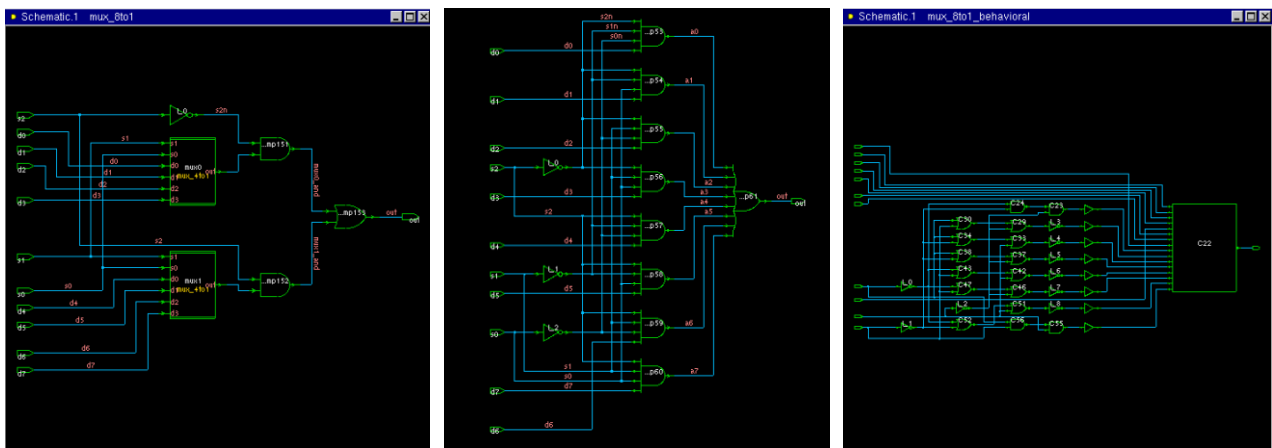
8to1 mux를 gate level로 구현하기 위해 (1) 4to1 mux를 gate level로 만든 뒤 탑 모듈에서 두 개를 instantiate하였고, (2) 처음부터 and, or, not만을 이용하는 두 가지 전략을 사용했다. Behavioral level에서는 case문을 사용해 8to1 mux를 구현했다.

a. Pre-Synthesis Simulation

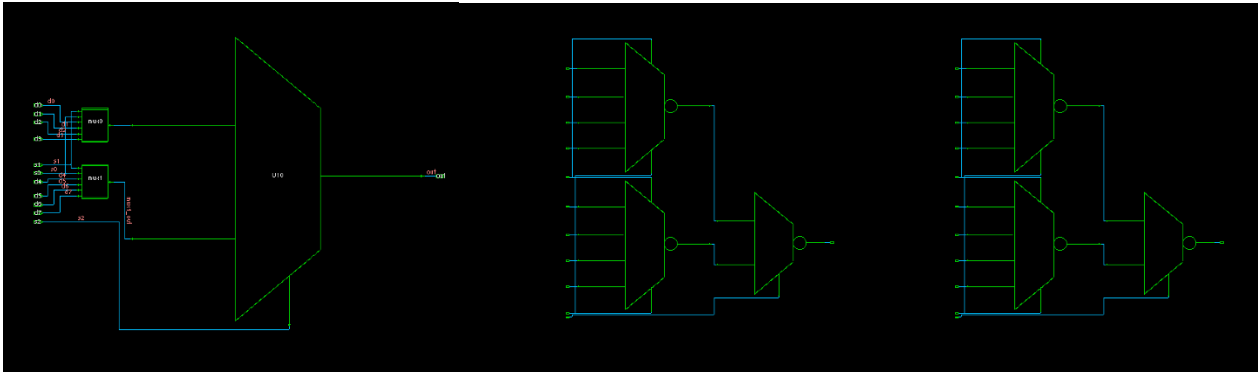


gate_out0이 방법 (2)를 사용한 gate level mux, gate_out1은 방법 (1)을 사용한 gate level mux, gate_out3은 behavioral level mux이다. Testbench를 보면 {i0, i1, i2, i3, i4, i5, i6, i7} = 8'b01010101로 값이 고정되어 있다. Select bit가 0부터 7까지 1씩 커지면 0과 1이 번갈아가면서 출력되는 것을 알 수 있다. 실제로 세 mux 모두 파형이 동일하고 정상 작동한다는 것을 확인할 수 있다.

b. Schematics



위의 그림은 왼쪽부터 (1) 4to1 mux를 이용한 8to1 mux, (2) and, or, not을 이용한 8to1 mux, (3) behavioral level 8to1 mux의 pre-synthesis schematic이다.



위의 그림은 왼쪽부터 (1) 4to1을 이용한 8to1 mux, (2) and, or, not을 이용한 8to1 mux, (3) behavioral level 8to1 mux의 post-synthesis schematic이다. 합성 후에는 (2)와 (3)이 동일한 schematic을 갖게 됨을 확인할 수 있다.

c. Area

Report : area	Report : area	Report : area
Design : mux_8to1	Design : mux_8to1_gate	Design : mux_8to1_behavioral
Version: Z-2007.03-SP4	Version: Z-2007.03-SP4	Version: Z-2007.03-SP4
Date : Sat Mar 21 16:58:24 2026	Date : Wed Mar 25 14:32:35 2026	Date : Sat Mar 21 16:56:48 2026
Library(s) Used:	Library(s) Used:	Library(s) Used:
lec25dsc25_SS (File: /home/admin/lib/lec25/lec25dsc25_SS.db)	lec25dsc25_SS (File: /home/admin/lib/lec25/lec25dsc25_SS.db)	lec25dsc25_SS (File: /home/admin/lib/lec25/lec25dsc25_SS.db)
Number of ports: 12	Number of ports: 12	Number of ports: 12
Number of nets: 14	Number of nets: 14	Number of nets: 14
Number of cells: 3	Number of cells: 3	Number of cells: 3
Number of references: 3	Number of references: 2	Number of references: 2
Combinational area: 340,069611	Combinational area: 298,599205	Combinational area: 298,599205
Noncombinational area: 0,000000	Noncombinational area: 0,000000	Noncombinational area: 0,000000
Net Interconnect area: undefined (No wire load specified)	Net Interconnect area: undefined (No wire load specified)	Net Interconnect area: undefined (No wire load specified)
Total cell area: 340,069611	Total cell area: 298,599213	Total cell area: 298,599213
Total area: undefined	Total area: undefined	Total area: undefined

왼쪽부터 (1) 4to1을 이용한 gate-level 8to1 mux, (2) and, or, not을 이용한 gate-level 8to1 mux, (3) behavioral level 8to1 mux의 post-synthesis area report이다. (1)의 Schematic이 나머지와 달랐기 때문에 면적 역시 다르게 나온 것을 확인할 수 있다. (2)와 (3)의 면적은 동일하다.

d. Result Analysis

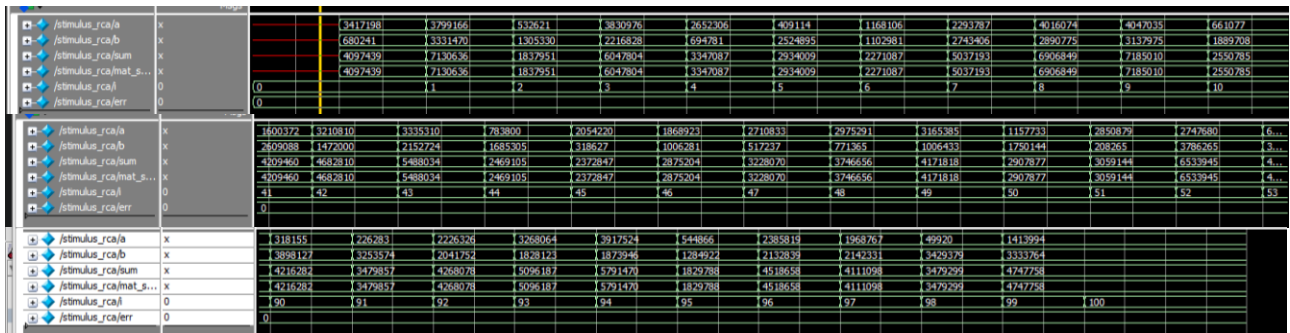
Gate level로 8to1 mux를 구현하기 위해서 처음에는 4to1 mux를 gate level로 만든 뒤 두 개의 4to1 mux를 instantiate했다. 하지만 실험의 의도와는 다르게 합성 후의 schematic이 동일하지 않았다. Schematic을 봤을 때 'mux0'과 'mux1'은 select bit가 하단으로 들어가는 mux가 아니라는 것을 확인할 수 있다. mux0과 mux1의 출력을 선택하는 회로는 2to1 mux로 제대로 합성됐지만, 디자인 컴파일러가 합성과정에서 4to1 mux를 infer하지 못한 것으로 보인다. 이 때문에 합성 후 면적 역시 and, or, not을 이용한 gate-level mux와 behavioral level mux와는 달랐다. 한편 이 두 8to1 mux는 합성 전 schematic이 달랐지만 같은 회로로 합성된 것을 볼 수 있으며 베릴로그 코딩 스타일에 관계없이 디자인 컴파일러가 최대한 회로를 최적화하기 위해 노력했음을 알 수 있다.

II. 22-bit Full Adder

22-bit ripple carry adder를 설계하여 타이밍 시뮬레이션을 통해 동작을 검증하고, FF를 추가한 뒤 합성하여 minimum clock cycle를 찾는 실험이다.

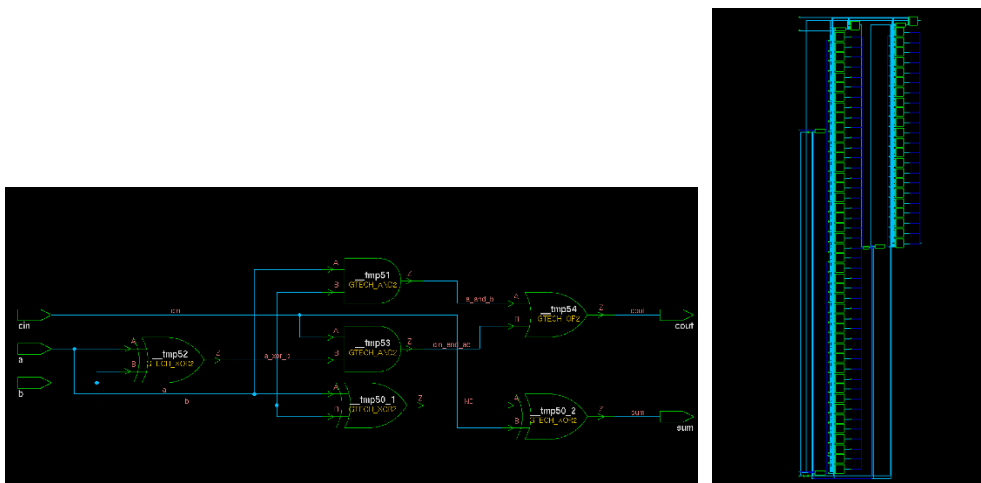
a. Pre-Synthesis Simulation

FF 없이 만든 22-bit ripple carry adder의 기능을 검증하기 위해 주어진 텍스트 파일에서 테스트 벡터를 읽고, 이 값들을 DUT에 입력하여 출력되는 값을 실제 덧셈값과 비교했다. 아래의 파형에서 보이는 a와 b가 operand이고, FA에서 출력되는 값인 sum을 mat_sum_cmp와 비교하여 차이가 있을 경우 err를 1씩 늘린다.



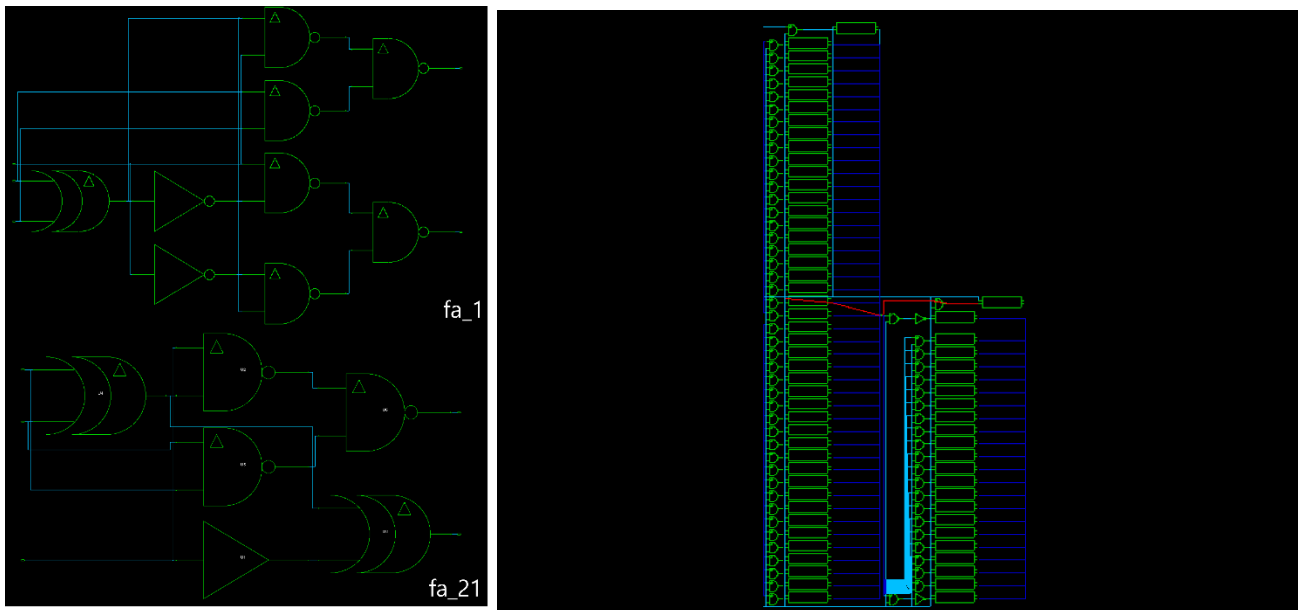
10ns가 지난 시점에서 a, b에 테스트 벡터의 성분들이 입력되고, 덧셈이 수행된다. i=100이 될 때까지 모든 sum과 mat_sum_cmp가 일치하고 err 역시 0이라는 것을 통해 22-bit ripple carry adder이 제대로 작동하는 것을 확인할 수 있다.

b. Schematics

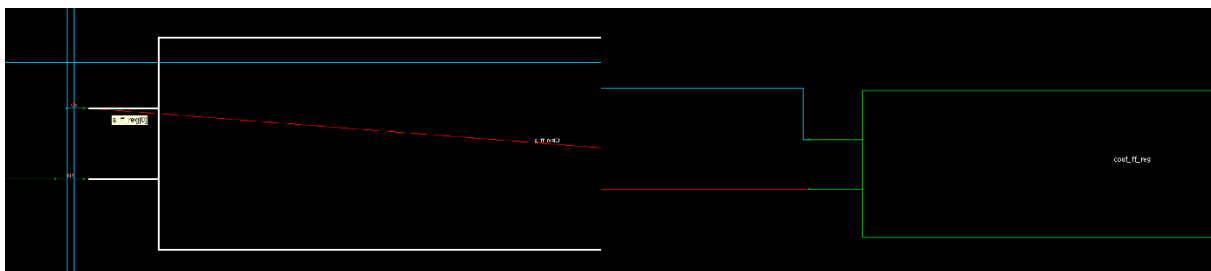


위 그림의 왼쪽부터 합성 전 1-bit FA와 합성 전 22-bit ripple carry adder의 schematic이다. 1-bit FA는 xor, and, or 게이트로 구현됨을 확인할 수 있다. 22-bit RCA의 경우 입력인 a, b, cin과

출력인 sum, cout이 1-bit FF에 저장되는데 schematic에서 이런 FF들을 볼 수 있다. 합성 후의 schematic은 각각 아래와 같다.



합성 후의 1-bit FA는 합성 전 schematic과 달리 NAND를 이용해서 회로를 구현했으며, fa instance 사이에서도 약간의 차이가 존재했다. 위 그림의 fa_1과 fa_21이 그 차이를 보여준다. 22-bit RCA의 경우 위 그림처럼 합성이 진행됐다. 빨간색 실선이 critical path를 나타낸다.



위의 그림은 critical path의 시점과 종점을 확대한 모습이다. 입력 a[0]이 저장되는 a_ff_reg[0]이 critical path의 시점이고, cout이 저장되는 cout_ff_reg가 종점이다. 따라서 마지막 carry bit가 critical path delay를 결정하는 것을 확인할 수 있다. Carry가 계산이 진행되며 propagate하는 경로가 critical path인 것이다.

c. Area & Timing

```

Report : area
Design : rca_22bit_ff
Version: Z-2007.03-SP4
Date   : Wed Mar 25 13:44:48 2026
*****
Library(s) Used:
    lec25dsc25_SS (File: /home/admin/lib/lec25/lec25dsc25_SS.db)

Number of ports:      70
Number of nets:       208
Number of cells:      139
Number of references: 8

Combinational area: 11968,819702
Noncombinational area: 10799,331665
Net Interconnect area: undefined (No wire load specified)

Total cell area: 22768,152344
Total area: undefined

add0/fa_gen[21].u_fa/U7/Q (nnd2s1) 0.16 5.01 r
add0/fa_gen[21].u_fa/U8/Q (nnd2s2) 0.10 5.11 f
add0/fa_gen[21].u_fa/cout (fa_1) 0.00 5.11 f
add0/cout (rca_22bit) 0.00 5.11 f
U140/Q (and2s2) 0.23 5.34 f
cout_ff_reg/DIN (dffs1) 0.00 5.34 f
data arrival time 5.34

clock clk (rise edge) 5.66 5.66
clock network delay (ideal) 0.00 5.66
cout_ff_reg/CLK (dffs1) 0.00 5.66 r
library setup time -0.32 5.34
data required time 5.34
-----
data required time 5.34
data arrival time -5.34
-----
slack (MET) 0.00

```

위 그림은 클럭 주기를 5.66ns로 설정하고 합성했을 때의 area 및 timing report이다. Non-negative slack이 되면서 타이밍을 만족하는 것을 볼 수 있다. 또한 worst timing path의 종점이 cout_ff_reg/DIN인 것으로 보아 carry propagation 경로가 critical path인 것도 알 수 있다. 종점에 신호가 도달한 시간(data arrival time)을 보면 5.34ns이므로, 이것이 critical path delay라고 볼 수 있다.

```

*****
Report : area
Design : rca_22bit_ff
Version: Z-2007.03-SP4
Date   : Wed Mar 25 13:39:17 2026
*****
Library(s) Used:
    lec25dsc25_SS (File: /home/admin/lib/lec25/lec25dsc25_SS.db)

Number of ports:      70
Number of nets:       207
Number of cells:      138
Number of references: 7

Combinational area: 12035,174904
Noncombinational area: 10799,331665
Net Interconnect area: undefined (No wire load specified)

Total cell area: 22834,506569
Total area: undefined

add0/fa_gen[21].u_fa/U6/Q (i1s3) 0.07 4.94 f
add0/fa_gen[21].u_fa/U5/Q (nnd2s2) 0.09 5.03 r
add0/fa_gen[21].u_fa/U1/Q (nnd2s2) 0.10 5.13 f
add0/fa_gen[21].u_fa/sum (fa_1) 0.00 5.13 f
add0/sum[21] (rca_22bit) 0.00 5.13 f
U72/Q (nnd2s2) 0.13 5.26 r
U71/Q (i1s3) 0.08 5.34 f
sum_ff_reg[21]/DIN (dffs1) 0.00 5.34 f
data arrival time 5.34

clock clk (rise edge) 5.65 5.65
clock network delay (ideal) 0.00 5.65
sum_ff_reg[21]/CLK (dffs1) 0.00 5.65 r
library setup time -0.32 5.33
data required time 5.33
-----
data required time 5.33
data arrival time -5.34
-----
slack (VIOLATED) -0.01

```

위 그림은 클럭 주기를 5.65ns로 설정하고 합성했을 때의 area 및 timing report이다. Area는 5.66ns일 때보다 소폭 증가한 것을 볼 수 있다. 디자인 컴파일러가 critical path를 줄이기 위해 면적을 늘렸지만 실패한 것이라고 생각할 수 있다. 또한 5.66ns일 때와는 다르게 worst timing path의 종점이 sum_ff_reg[21]/DIN으로, Cout이 아니라 마지막 sum 비트임을 알 수 있다. 마지막 sum[21]이 만들어지기 위해서는 그 전 단계에서 만들어진 carry bit가 필요하기 때문에 sum[21] 역시 carry propagation의 영향을 받으며, 합성 경우에 따라서 최종 sum 비트가 critical path를 결정지을 수도 있다는 것을 확인할 수 있다.

d. Result Analysis

합성을 위해 클럭 조건을 5.66ns로 줬을 때는 timing이 met되는 반면, 5.65ns부터는 slack이 음수로 타이밍 위반이 발생했다. 다시 5.66ns로 합성했을 때는 타이밍 위반이 발생하지 않았으므로 minimum clock period는 5.66ns, F_{max} 는 $\frac{1}{5.66 \times 10^{-9}} = 176.7\text{MHz}$ 이다. 이 경우 22-bit RCA에서 critical path는 cout, 즉 carry propagation path임을 확인할 수 있었다.